Now this is interesting because our newly thrown exception has in fact caused an E_ERROR to be emitted, which is why we see the PHP Fatal error. The error occurs because nothing caught the exception. This makes sense because exceptions are intended to change program flow, and if we don't have any alternative code for PHP to execute, then it rightly stops, which brings us to our next sub-topic…

Catching Exceptions

PHP's language construct for exception handling is the same as many others: try, catch, and finally. When you know that certain code is able to throw an exception, wrap it in a try { } block. A try block must then have at least one catch or finally block attached to it (to avoid a fatal E_ERROR). If a line of code inside the try block throws an exception (or calls something that causes an exception to be thrown), then execution immediately switches to the catch block, and the rest of the try block is abandoned:

```
function boom()
{
    throw new Exception("Emergency... there's an emergency going on.");
    print "I'll never be reached...";
}

try {
    boom();
} catch (Exception $e) {
    print $e->getMessage() . PHP_EOL;
}

print "It's still going on.";

Output:
Emergency... there's an emergency going on.
It's still going on.
```

The construct catch (Exception $e) tells PHP that if the thrown exception is an instance of Exception, then it should execute the code inside the braces. At this point, catch works a bit like a function call, with the thrown object passed as an argument $e, which the code inside the braces can then make use of. Our catch block code invokes a special function that "belongs" to the object – a *method* – using the -> operator.